

week2 연습과제

≡ 다중 선택

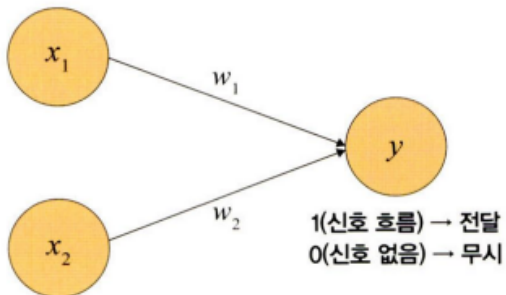
week2

4장 딥러닝 시작

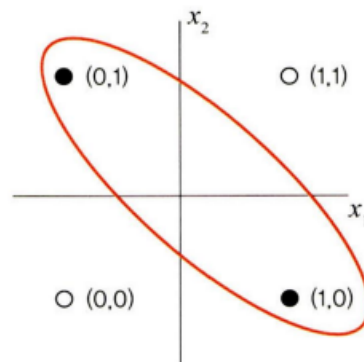
4.1 인공 신경망의 한계와 딥러닝 출현

단층 퍼셉트론

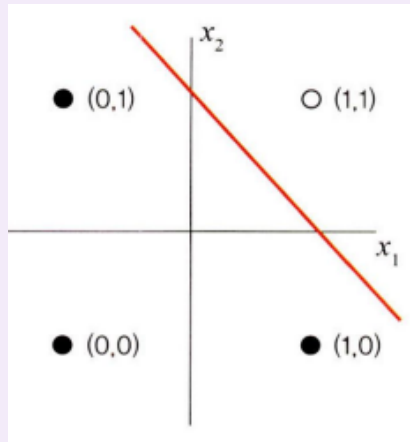
- 개념



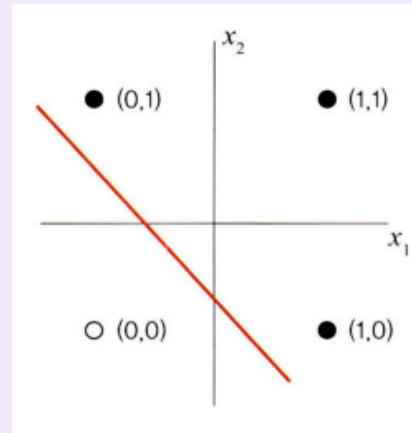
- 신경망(딥러닝)의 기원이 되는 알고리즘
- 다수의 신호를 입력으로 받아 하나의 신호로 출력
 - '흐른다/안 흐른다(1 또는 0)'는 정보를 앞으로 전달
- 한계
 - XOR 게이트는 데이터가 비선형적으로 분리됨 → 단층 퍼셉트론에서는 학습 불가



▼ AND 게이트 & OR 게이트



AND 게이트: 모든 입력이 '1'일때 작동



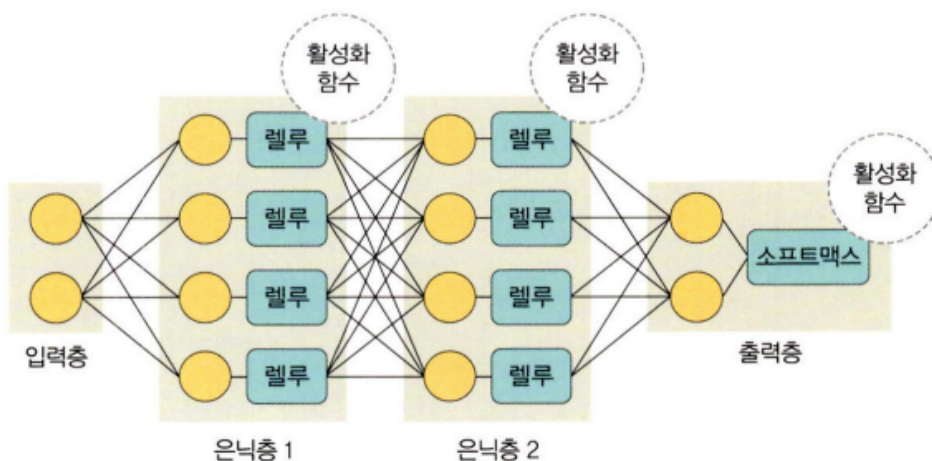
OR 게이트: 둘 중 하나라도 '1'이면 작동

• 극복

- 입력층과 출력층 사이에 **하나 이상의 중간층(은닉층)**을 두어 **비선형적으로** 분리되는 데이터에 대해서도 학습이 가능하도록 **다층 퍼셉트론**을 고안
 - 은닉층 여러개 있는 신경망 → 심층 신경망(딥러닝)

4.2 딥러닝 구조

4.2.1 딥러닝 용어



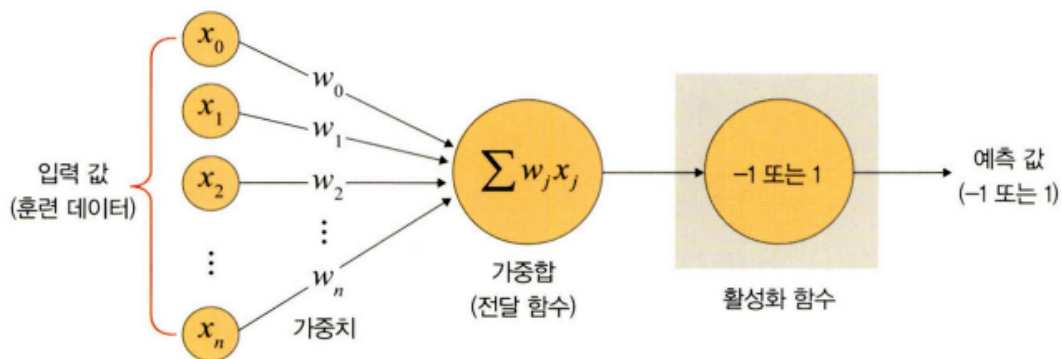
구분	구성 요소	설명
층 (Layer)	입력층 (Input Layer)	외부 데이터를 입력받아 신경망으로 전달하는 층
	은닉층 (Hidden Layer)	이전 노드의 입력을 받아 가중합을 계산하고, 활성화 함수를 거쳐 다음 층으로 전달하는 층
	출력층 (Output Layer)	신경망의 연산이 완료된 최종 예측 및 결과값을 반환하는 층
변수 및 연산	가중치 (Weight)	노드 간 연결의 강도를 나타내는 매개변수

구분	구성 요소	설명
	편향 (Bias)	가중합에 추가되어 뉴런의 활성화 기준(임계값)을 조절하는 상수
	가중합 / 전달 함수 (Weighted Sum)	입력값과 각 가중치의 곱을 모두 더한 값
함수 (Function)	활성화 함수 (Activation Function)	가중합을 입력받아 비선형성을 추가하고 최종 출력 신호를 결정하는 함수
	손실 함수 (Loss Function)	모델의 예측값과 실제값 사이의 오차를 측정하여 가중치 학습의 기준을 제공하는 함수

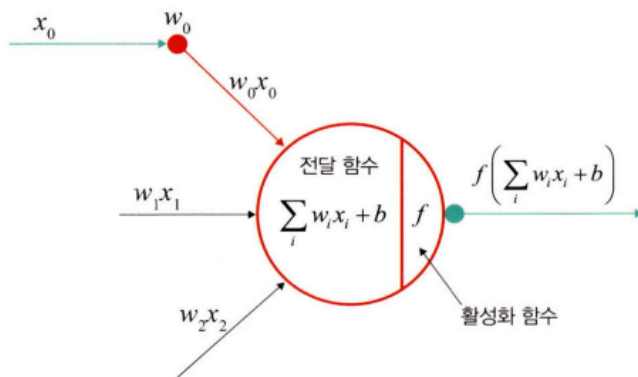
1) 가중치

: 입력 값이 연산 결과에 미치는 영향력을 조절하는 요소

→ 입력 값(x)이 아무리 크더라도 가중치(w)가 0에 가깝다면 wx 값은 0에 가까운 값이 됨



2) 가중합(전달함수)



- 입력값과 가중치의 곱을 모두 더한 후, 편향을 추가하여 활성화 함수로 전달하는 연산 과정
 - 각 노드에서 계산된 이 합계가 다음 단계인 활성화 함수로 그대로 전달(Transfer)되기 때문에 **전달 함수**라고도 부름
- 계산 수식

$$\sum_{i=1}^n w_i x_i + b$$

- x: 입력값 (Input Signal)
- w: 가중치 (Weight - 연결 강도)
- b: 편향 (Bias - 활성화 조절 상수)

• 연산 흐름

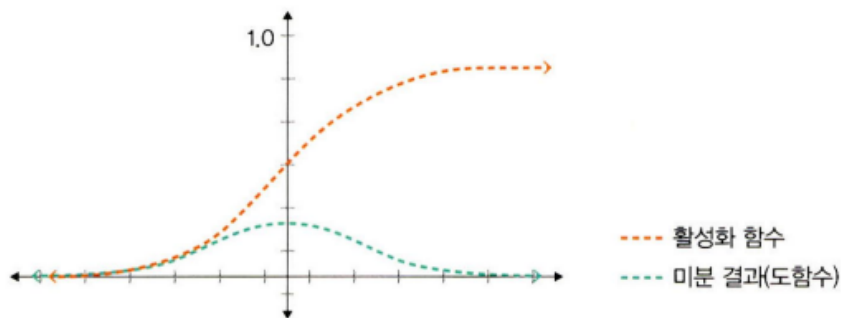
[입력값 x] × [가중치 w] → [전체 합산 $\sum(wx)$ + 편향 b] → [활성화 함수 f] → [최종 출력]

└─ 전달 함수(가중합) ─┘

3) 활성화 함수

: 전달 함수에서 전달받은 값을 출력할 때 일정 기준에 따라 출력 값을 변화시키는 **비선형 함수**

• 시그모이드 함수



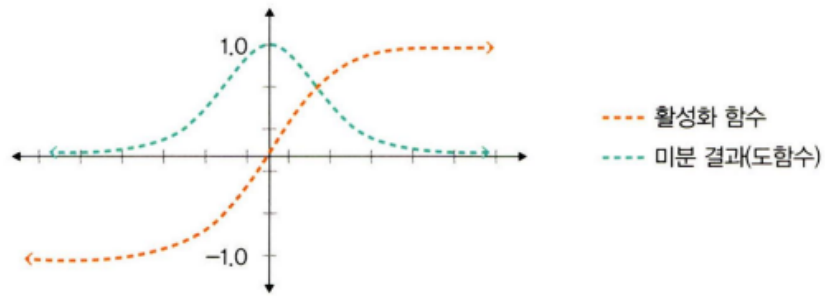
- 수식:

$$f(x) = \frac{1}{1 + e^{-x}}$$

- 특징

- 선형 함수의 결과를 **0~1 사이의 비선형 값**으로 변환
- 로지스틱 회귀 등에서 출력값을 **확률적으로 표현**하는 데 사용
- 과거에는 널리 사용되었으나, **기울기 소멸 문제**로 인해 최근 딥러닝 모델의 은닉층에서는 사용 빈도가 낮음
 - 깊은 신경망에서 역전파 시 기울기가 점점 작아져 학습이 어려워지는 문제

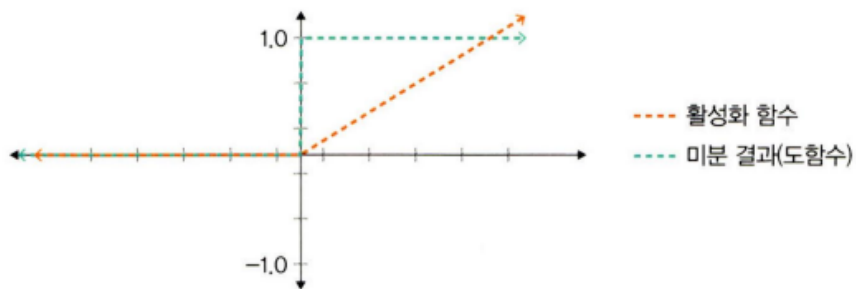
• 하이퍼볼릭 탄젠트 함수



◦ 특징

- 선형 함수의 결과를 **1~1 사이의 비선형 값**으로 변환
- 시그모이드 함수의 출력값이 0이 아닌 양수로 편향되는 문제를 완화
 - 출력값의 평균이 0에 가까워지는 **제로 센터링(zero-centering)** 효과
- 시그모이드 함수와 마찬가지로 **기울기 소멸 문제**가 발생할 수 있음

• 렐루(ReLU) 함수



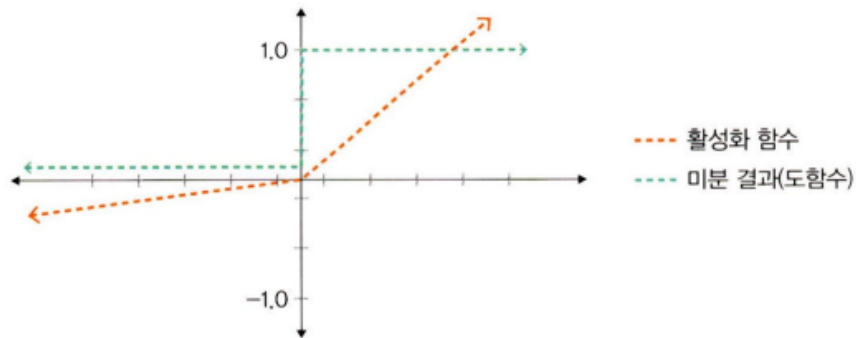
◦ 수식

$$f(x) = \max(0, x)$$

◦ 특징

- 입력값이 음수이면 **0**, 양수이면 **입력값 그대로** 출력
- 계산이 단순하여 학습 속도가 빠르고, **기울기 소멸 문제를 완화**할 수 있음
- 일반적으로 **은닉층의 활성화 함수**로 사용
- 하이퍼볼릭 탄젠트 함수보다 학습 속도가 빠른 것으로 알려짐
- 단, 음수 입력에 대해 항상 0을 출력하므로 해당 뉴런이 더 이상 학습하지 못하는 **죽은 ReLU(Dead ReLU) 문제**가 발생할 수 있음
⇒ **Leaky ReLU** 등을 사용하여 보완

• 리키 렐루(Leaky RELU) 함수



◦ 특징

- 음수 입력에 대해서도 **0이 아닌 작은 값**을 출력
 - 예: `0.001x`
- 음수 구간에서도 작은 기울기를 유지하여 **죽은 ReLU 문제를 완화**
- ReLU의 장점을 유지하면서 음수 입력에 대한 학습 가능성을 확보

• 소프트맥스 함수

◦ 특징

- 여러 출력값을 **0~1 사이의 확률값**으로 변환
- 모든 출력값의 총합이 **항상 1**이 되도록 정규화
- 주로 **다중 클래스 분류 문제의 출력층**에서 사용

◦ 수식

$$y_k = \frac{e^{a_k}}{\sum_{i=1}^n e^{a_i}}$$

- a_k : 출력층 k번째 노드의 입력값(가중합)
- n : 출력층의 총 노드 수 (전체 클래스 개수)
- e : 자연상수 (Euler's number)
- y_k : k번째 클래스에 속할 확률값

• 예시: ReLU 함수와 softmax 함수 구현

```
import torch
import torch.nn as nn

class Net(nn.Module):
```

```

def __init__(self, n_feature, n_hidden, n_output):
    super(Net, self).__init__()

    # 1. 은닉층: (입력 특징 수 -> 은닉층 노드 수) 선형 변환 ( $wx + b$ ) (전달 함수)
    self.hidden = nn.Linear(n_feature, n_hidden)

    # 2. 은닉층 활성화 함수: ReLU
    # inplace=True: 메모리 연산 최적화를 위해 입력을 직접 수정
    self.relu = nn.ReLU(inplace=True)

    # 3. 출력층: (은닉층 노드 수 -> 최종 출력 클래스 수) 선형 변환 ( $wx + b$ ) (전달함수)
    self.out = nn.Linear(n_hidden, n_output)

    # 4. 출력층 활성화 함수: Softmax
    # dim=1: (batch_size, n_output) 형태의 입력에서 각 데이터별 클래스 확률의 합이 1이 되도록 계산
    self.softmax = nn.Softmax(dim=1)

def forward(self, x):
    # 입력 데이터 x가 순전파(Forward propagation) 과정을 거치는 흐름

    x = self.hidden(x) # 선형 변환 (가중합 계산)
    x = self.relu(x) # 은닉층 비선형 활성화 함수 적용
    x = self.out(x) # 출력층 선형 변환
    x = self.softmax(x) # 다중 클래스 분류를 위한 확률값 변환 (출력 합 = 1)

    return x

```

4) 손실 함수

: 모델의 예측값이 실제 정답과 얼마나 차이가 나는지 **측정하는 함수**

- 손실값이 클수록 **예측이 부정확함**을 의미
- 손실값이 0에 가까울수록 **정답에 가까운 예측**을 의미
- 경사 하강법에서는 손실 함수의 기울기를 이용하여 **손실이 감소하는 방향으로 가중치를 업데이트**함

1. 평균 제곱 오차 (Mean Squared Error, MSE) → 주로 회귀 문제

: 실제값과 예측값의 차이를 **제곱한 뒤 평균**을 구한 손실 함수

- 수식

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- **y_i**: 실제값 (정답 레이블)
- **ŷ_i**: 예측값 (신경망이 추정한 값)
- **n**: 데이터 개수 (샘플 수)
- 특징
 - 실제값과 예측값의 차이가 클수록 **손실값이 크게 증가**
 - 손실값이 작을수록 **예측 성능이 좋음**을 의미
 - 주로 **회귀 문제의 손실 함수**로 사용
 - 오차를 제곱하므로 **큰 오차에 더 큰 페널티**를 부여
- PyTorch 사용법

```
import torch

loss_fn = torch.nn.MSELoss(reduction='sum')

y_pred = model(x)
loss = loss_fn(y_pred, y)
```

▼ 설명

- **MSELoss**: 평균 제곱 오차를 계산하는 손실 함수
- **reduction='sum'**: 각 데이터의 손실을 **합산**하여 반환
 - 기본값은 **'mean'** 으로, 손실의 평균을 반환

2. 크로스 엔트로피 오차 (Cross Entropy Error, CEE) → 주로 분류 문제

: 실제 정답과 모델이 예측한 **확률 분포의 차이**를 측정하는 손실 함수

- 수식 (다중 클래스 분류)

$$CrossEntropy = - \sum_{k=1}^n y_k \log(\hat{y}_k)$$

- **y_k**: 정답 레이블 (원-핫 인코딩 적용 시 정답 클래스만 1, 나머지는 0)
- **ŷ_k**: k번째 클래스에 대한 예측 확률 (소프트맥스 등을 거친 출력값)
- **n**: 클래스 개수
- 특징
 - 정답 클래스의 예측 확률에 따라 손실값이 결정됨

- 정답 클래스에 **높은 확률**을 부여하면 손실이 작아짐
- 정답 클래스에 **낮은 확률**을 부여하면 손실이 커짐
- 분류 문제에서 **모델의 확률 예측을 학습**하는 데 적합

• PyTorch 사용법

```
import torch
import torch.nn as nn

loss = nn.CrossEntropyLoss()

input = torch.randn(5, 6, requires_grad=True)
target = torch.empty(5, dtype=torch.long).random_(6)

output = loss(input, target)
output.backward()
```

▼ 설명

- **input**: 모델의 출력값인 **로짓(logit)**
 - 크기: (배치 크기, 클래스 개수) → (5, 6)
 - **CrossEntropyLoss** 내부에서 소프트맥스와 로그 연산을 함께 수행하므로 **소프트맥스를 별도로 적용하지 않음**
- **target**: 실제 정답 클래스의 인덱스
 - **dtype=torch.long** 사용
 - Tensor(텐서)의 데이터 타입을 '64비트 정수형(64-bit integer)'으로 지정
 - 예: 0, 1, 2, 3, 4, 5



경사 하강법 (Gradient Descent)

: 손실 함수의 **순간 기울기(미분값)**와 **학습률(learning rate)**을 이용하여 가중치를 업데이트하는 방법

- 손실 함수의 기울기를 계산하여 **오차가 감소하는 방향으로 이동**
- 학습률 η 는 한 번의 학습에서 가중치를 **얼마나 변화시킬지 결정하는 상수**
- 기본적인 가중치 업데이트 수식

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}$$

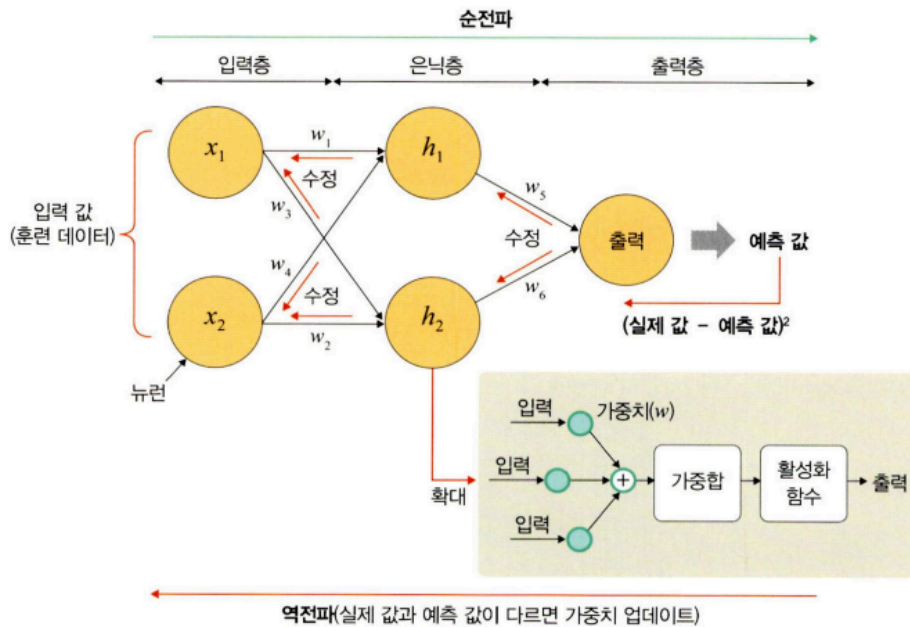
- w: 가중치 (Weight)
- η : 학습률 (Eta / Learning Rate)

- L: 손실 함수 (Loss Function)
- $\partial L / \partial w$: 가중치에 대한 손실 함수의 편미분값 (기울기 / Gradient)

4.2.2 딥러닝 학습

• 신경망의 학습 과정

: 신경망은 순전파 → 손실 계산 → 역전파 → 가중치 업데이트 과정을 반복하며 학습함



1. 순전파 (Feedforward)

: 입력 데이터를 신경망에 전달하여 예측값을 계산하는 과정

- 입력 데이터가 입력층 → 은닉층 → 출력층 순서로 전달됨
- 각 뉴런은 이전 층에서 전달받은 정보에 가중합과 활성화 함수를 적용하여 다음 층으로 전달
- 모든 층의 계산이 완료되면 최종 예측값이 출력됨
- 입력 데이터 → 가중합 → 활성화 함수 → 다음 층 → 출력값

2. 손실 계산 (Loss Calculation)

: 모델의 예측값과 실제값의 차이를 손실 함수로 계산하는 과정

- 예측값과 실제값의 차이를 손실(loss) 또는 오차(error)라고 함
- 손실값이 작을수록 예측이 실제값에 가까움을 의미
- 이상적인 손실값은 0
- 계산된 손실을 바탕으로 가중치를 조정하여 손실을 줄이는 방향으로 학습함(역전파)

3. 역전파 (Backpropagation)

: 손실 함수의 기울기를 이용하여 각 가중치가 손실에 얼마나 영향을 미쳤는지 계산하는 과정

- 손실이 계산되면 출력층에서 입력층 방향으로 역순으로 기울기를 전파
- 각 가중치에 대한 손실 함수의 기울기를 미분(편미분)하여 계산
- 계산된 기울기는 해당 가중치가 손실에 미친 상대적인 영향을 나타냄
- 역전파를 통해 계산된 기울기는 이후 가중치 업데이트에 사용됨
- 출력층 → 은닉층 → 입력층 순서로 기울기 계산

4. 가중치 업데이트 (Weight Update)

: 역전파로 계산한 기울기를 이용하여 가중치를 조정하는 과정

- 경사 하강법을 이용하여 손실이 감소하는 방향으로 가중치를 업데이트
- 기본적인 가중치 업데이트 수식

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}$$

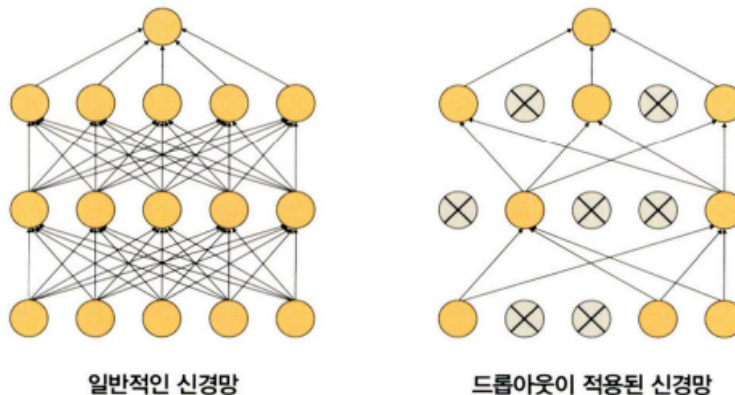
- 업데이트된 가중치는 다음 순전파에서 다시 사용됨

4.2.3 딥러닝의 문제점과 해결방안

→ 은닉층 개수가 많을수록 훈련 데이터의 분류가 잘 됨 but 여러 문제 발생

1) 과적합

- 훈련 데이터의 노이즈까지 과하게 학습하여 실제 데이터에 대한 오차가 증가
 - 훈련 데이터에서의 성능은 좋지만 검증 데이터에서의 성능은 좋지 않을 수 있음
- 대안
 - 드롭아웃: 학습 과정 중 임의로 일부 노드들을 학습에서 제외시킴



```

import torch
import torch.nn as nn
import torch.nn.functional as F

class DropoutModel(nn.Module):

    def __init__(self):
        super(DropoutModel, self).__init__()

        # 층 정의
        self.layer1 = nn.Linear(784, 1200) # 입력층 -> 은닉층 1
        self.dropout1 = nn.Dropout(p=0.5) # 50% 확률로 드롭아웃

        self.layer2 = nn.Linear(1200, 1200) # 은닉층 1 -> 은닉층 2
        self.dropout2 = nn.Dropout(p=0.5) # 50% 확률로 드롭아웃

        self.layer3 = nn.Linear(1200, 10) # 은닉층 2 -> 출력층

    def forward(self, x):
        # 순전파 흐름
        x = F.relu(self.layer1(x))
        x = self.dropout1(x) # 은닉층 1 출력에 드롭아웃 적용

        x = F.relu(self.layer2(x))
        x = self.dropout2(x) # 은닉층 2 출력에 드롭아웃 적용

        return self.layer3(x) # 최종 출력층 전달 (Logits 반환)

```

2) 기울기 소멸 문제

- 은닉층이 많은 신경망에서 주로 발생
- 출력층에서 은닉층으로 전달되는 오차가 크게 줄어들어(기울기가 0에 가까워져) 학습이 되지 않는 현상
- 대안
 - 시그모이드 함수 / 하이퍼볼릭 탄젠트 → X
 - 렐루 함수 → O

▼ 1. 기울기 소멸(Vanishing Gradient)이 일어나는 핵심 이유

→ 역전파 연산 과정에서 **연쇄 법칙(Chain Rule)**에 의해 1보다 작은 미분값들이 계속 곱해지기 때문

1. 시그모이드(Sigmoid)나 하이퍼볼릭탄젠트(tanh)의 한계

- 시그모이드 함수 $f(x) = \frac{1}{1+e^{-x}}$ 의 미분값 $f'(x)$ 의 최대값은 **0.25** (아무리 커봤자 0.25 이하)
- tanh 함수 미분값의 최대값도 **1.0** 이하이며, 양 끝으로 갈수록 0에 수렴

2. 은닉층이 깊어질 때 발생하는 일

역전파는 출력층부터 입력층 방향으로 각 층의 활성화 함수 미분값을 계속 곱해나감

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial a_n} \cdot \frac{\partial a_n}{\partial z_n} \cdot \frac{\partial z_n}{\partial a_{n-1}} \cdots \frac{\partial a_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_1} = \frac{\partial L}{\partial y} \cdot f'(z_n) \cdot w_n \cdots f'(z_1) \cdot x_1$$

- $f'(z)$ 값이 0.25라고 가정하고 층이 10개만 깊어져도:
 $0.25^{10} \approx 0.00000095$
- 1보다 작은 소수가 거듭 곱해지면서 **입력층에 가까운 앞쪽 은닉층으로 갈수록 기울기가 0으로 수렴**
- **결과:** 앞쪽 층의 가중치가 업데이트되지 않아 딥러닝 모델이 제대로 학습하지 못함

▼ 2. ReLU는 왜 음수를 0으로 죽여버릴까?

- ReLU(Rectified Linear Unit)는 $f(x) = \max(0, x)$ 형태
 - 양수 영역 ($x > 0$): 미분값이 무조건 **1**
 - 역전파 시 1을 곱하므로 기울기가 줄어들지 않고 그대로 전달되어 **기울기 소멸 문제를 깔끔하게 해결**
 - 음수 영역 ($x \leq 0$): 출력도 0, 미분값도 **0**
- **왜 굳이 음수를 0으로 죽일까?**
 1. **희소성(Sparsity) 확보 (핵심 이유)**
 - 신경망의 모든 노드가 동시에 활성화되면 연산량이 많아지고 **과적합(Overfitting)**이 일어나기 쉬움
 - 음수 입력에 대해 0을 출력하면 **일부 뉴런을 오프(OFF) 상태**로 만들어, 데이터의 중요한 특징에만 반응하는 효율적이고 가벼운 딥러닝 모델을 만들 수 있음
 2. **연산 단순화**
 - 단순한 비교 연산(max)과 기울기 1 또는 0만 사용하므로 시그모이드 같은 지수 연산(e^{-x})에 비해 학습 속도가 엄청나게 빠릅니다.
- 그럼 **음수가 0이 되어 뉴런이 아예 죽는 문제(Dying ReLU)**는?"
 - 음수 영역에서는 기울기가 0이 되어 가중치 업데이트가 영원히 멈추는 **Dying ReLU** 현상이 발생 가능
 - → 음수 영역에서도 0 대신 아주 작은 기울기(예: 0.01)를 남겨두는 **Leaky ReLU**나 **ELU** 같은 변형 활성화 함수가 등장

3) 성능 저하 문제

- 경사 하강법은 손실 함수의 비용이 최소가 되는 지점을 찾을 때까지 기울기가 낮은 쪽으로 계속 이동시키는 과정을 반복하는데, 이때 성능이 나빠지는 문제가 발생



구분	배치 경사 하강법 (BGD)	확률적 경사 하강법 (SGD)	미니 배치 경사 하강법 (Mini-batch GD)
개념	전체 데이터셋을 한 번에 사용하여 기울기 계산	단 1개의 데이터(임의 추출)를 사용하여 기울기 계산	전체 데이터를 일정한 크기(Batch Size)로 나누어 계산
수식	$W \leftarrow W - \alpha \nabla J(W, b)$	$W \leftarrow W - \alpha \nabla J(W, b, x^{(i)}, y^{(i)})$	$W \leftarrow W - \alpha \nabla J(W, b, x^{(i:i+m)}, y^{(i:i+m)})$
장점	- 이동 궤적이 매우 매끄럽고 안정적 - 전역 최솟값(Global Minimum)에 안정적으로 수렴	- 계산량이 적어 1 step당 속도가 매우 빠름 - 국소 최솟값(Local Minimum)을 쉽게 탈출함	- BGD보다 빠른 속도 + SGD보다 안정적인 궤적 - GPU 병렬 연산에 가장 최적화되어 실제 딥러닝에서 표준 사용
단점	- 전체 데이터를 메모리에 올려야 하므로 메모리 부족 발생 위험 - 데이터가 크면 1 step 계산 시간이 너무 오래 걸림	- 파라미터 업데이트 궤적이 튀어 불안정함(Zigzag) - 최댓값/최솟값 주변에서 헤맬 수 있음	미니 배치 크기(Batch Size)라는 하이퍼파라미터를 추가로 설정해야 함

```
import torch
from torch.utils.data import DataLoader, Dataset
```

1. 커스텀 데이터셋 정의 (Dataset 클래스 상속)

```
class CustomDataset(Dataset):
```

```
    def __init__(self):
```

```
        # 학습에 사용할 전체 데이터 정의
```

```
        self.x_data = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

```
        self.y_data = [[12], [18], [11]]
```

```
    def __len__(self):
```

```
        # 데이터셋의 총 데이터 개수 반환
```

```
        return len(self.x_data)
```

```
def __getitem__(self, idx):
    # 인덱스(idx)에 해당하는 단일 데이터를 PyTorch FloatTensor 타입으로 변환
    # 하여 반환
    x = torch.FloatTensor(self.x_data[idx])
    y = torch.FloatTensor(self.y_data[idx])
    return x, y

# 2. 데이터셋 인스턴스 생성
dataset = CustomDataset()

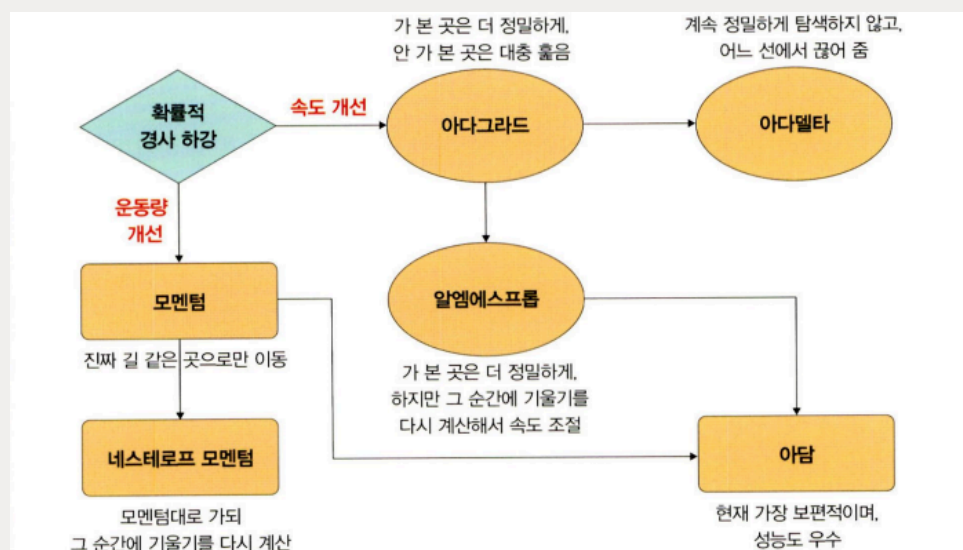
# 3. 데이터로더 정의 (미니 배치 생성 및 셔플)
dataloader = DataLoader(
    dataset, # 불러올 데이터셋 객체
    batch_size=2, # 미니 배치 크기 (보통 2의 거듭제곱 수인 32, 64, 128 등
    # 사용)
    shuffle=True, # 에포크(Epoch)마다 데이터를 무작위로 섞어서 모델의 과적합
    # 방지
)
```

▼ 📌 옵티마이저 (Optimizer)



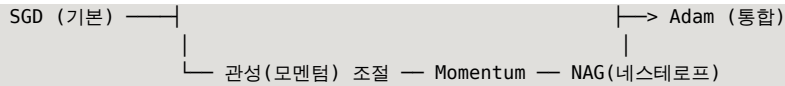
확률적 경사 하강법(SGD)의 파라미터 업데이트 궤적이 흔들리고 불안정한 문제를 해결하기 위해, 학습률(속도)과 관성(모멘텀/운동량)을 조절하여 최적의 가중치를 찾는 알고리즘

• 옵티마이저 유형



• 옵티마이저 발전 계보

└─ 학습률(속도) 조절 ─ Adagrad ─ RMSprop ─ Adadelta



1. 학습률(Step Size)을 조정하는 방법

1) Adagrad (Adaptive Gradient)

- : 가중치(파라미터)의 업데이트 횟수와 변화량에 따라 학습률을 유동적으로 조절
- **원리**: 많이 변화한 가중치는 최적값에 가까워졌다고 판단하여 학습률을 작게 하고, 적게 변화한 가중치는 학습률을 크게 설정(→ 타원형 생각하기)
- **수식**:
 - $G^{(t)} = G^{(t-1)} + (\nabla E(w^{(t)}))^2$
 - 파라미터마다 다른 학습률
 - 이전 G 값의 누적
 - $\nabla E(w)$ (**기울기 / Gradient**): 가중치 w 에 대한 손실 함수 E 의 편미분값
 - $w^{(t+1)} = w^{(t)} - \frac{\eta}{\sqrt{G^{(t)} + \epsilon}} \nabla E(w^{(t)})$
- **한계점**: 학습이 진행될수록 G 값이 무한히 커져 학습률이 0에 수렴하여, 결국 **가중치 업데이트가 멈추는 문제**가 발생
- `optimizer = torch.optim.Adagrad(model.parameters(), lr=0.01)`

2) RMSprop

- : Adagrad의 G(t)가 무한히 커지는 문제를 해결하기 위해 **지수 이동 평균(EMA)** 기법을 도입한 방법
- **원리**: 과거의 모든 기울기를 똑같이 누적하지 않고, 최근 기울기 변화량을 더 크게 반영(감마(γ) 사용)하여 학습률이 0으로 급격히 떨어지는 것을 방지
- **수식**:
 - $G^{(t)} = \gamma G^{(t-1)} + (1 - \gamma) (\nabla E(w^{(t)}))^2$
 - γ : "얼마나 잊어버릴 것인가(비중)"를 결정하는 숫자
 - $w^{(t+1)} = w^{(t)} - \frac{\eta}{\sqrt{G^{(t)} + \epsilon}} \nabla E(w^{(t)})$
- `optimizer = torch.optim.RMSprop(model.parameters(), lr=0.01)`

3) Adadelata (Adaptive Delta)

- : Adagrad의 학습률 수렴 문제를 해결하기 위해, 하이퍼파라미터인 학습률 η 자체를 **가중치 변화량의 누적값(D)으로 대체**한 방법
- **특징**: 학습률(lr) 하이퍼파라미터를 따로 설정하지 않아도 작동

- 수식:

- $G^{(t)} = \gamma G^{(t-1)} + (1 - \gamma) (\nabla E(w^{(t)}))^2$

- $D^{(t)} = \gamma D^{(t-1)} + (1 - \gamma) (\Delta w^{(t)})^2$

- 가중치 변화량(Δw)의 누적 가중평균

- 별도의 학습률 설정 없이도 수식 단위가 깔끔하게 맞물려 스스로 스케일을 조절하도록

- $w^{(t+1)} = w^{(t)} - \frac{\sqrt{D^{(t-1)} + \epsilon}}{\sqrt{G^{(t)} + \epsilon}} \nabla E(w^{(t)})$

- `optimizer = torch.optim.Adadelta(model.parameters(), lr=1.0)`

2. 운동량(Momentum)을 조정하는 방법

1) Momentum

- : 경사 하강법에 **물리학의 관성(Momentum) 개념**을 도입한 방식

- **원리:** 이전 단계에서 이동했던 방향과 속도(v)를 일정 비율 반영 → 진동(지그재그 현상) ↓ + 최적값 방향으로 빠르게 이동 + Local Minimum에 빠지는 것 방지

- 수식:

- $v^{(t)} = \gamma v^{(t-1)} + \eta \nabla E(w^{(t)})$

- $w^{(t+1)} = w^{(t)} - v^{(t)}$

- `optimizer = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9)`

2) NAG (Nesterov Accelerated Gradient / 네스테로프 모멘텀)

- : 모멘텀의 단점(관성이 너무 커서 최적값을 지나쳐버리는 오버슈팅 현상)을 보완한 방법

- **원리:** 현재 위치에서 기울기를 먼저 계산X → **모멘텀 방향으로 미리 일단 이동한 위치에서 기울기를 재계산**하여 이동 방향을 정확하게 수정(제동 장치 역할)

1. 관성(이전 속도)만 가지고 **미리 앞으로 스윙 이동 (예상 도착지).**

2. 그 **예상 도착 지점의 기울기**를 미리 살펴봄

3. 만약 그곳이 다시 올라가는 오르막길(기울기 반전) → 미리 브레이크를 걸어 속도를 줄

- 수식:

- $v^{(t)} = \gamma v^{(t-1)} + \eta \nabla E(w^{(t)} - \gamma v^{(t-1)})$

- $w^{(t+1)} = w^{(t)} - v^{(t)}$

- `optimizer = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9, nesterov=True)`

3. 속도 + 운동량을 모두 결합한 방법

1) Adam (Adaptive Moment Estimation)

- : Momentum(1차 모멘트: 관성)과 RMSprop(2차 모멘트: 적응형 학습률)의 장점을 하나로 결합한 옵티마이저
- 특징: 딥러닝 분야에서 **가장 보편적으로 사용, 성능 우수**
- 수식:
 - $v^{(t)} = \gamma_1 v^{(t-1)} + (1 - \gamma_1) \nabla E(w^{(t)})$
 - $G^{(t)} = \gamma_2 G^{(t-1)} + (1 - \gamma_2) (\nabla E(w^{(t)}))^2$
 - $w^{(t+1)} = w^{(t)} - \frac{\eta}{\sqrt{G^{(t)} + \epsilon}} v^{(t)}$
- `optimizer = torch.optim.Adam(model.parameters(), lr=0.001)`

4.2.4 딥러닝을 사용할 때 이점

1) 특성 추출

- : 사람이 인지하는 데이터를 컴퓨터가 분석할 수 있는 패턴/규칙(벡터 형태)으로 변환하는 작업
- 전통적 머신러닝 (SVM, Naive Bayes, 로지스틱 회귀 등):
 - 사람이 직접 특성을 정의하고 추출
 - 데이터가 수집된 **도메인 분야(의료, 제조 등)에 대한 높은 전문 지식**과 복잡한 작업이 필수적
- 딥러닝 (Deep Learning):
 - 은닉층(Hidden Layer)을 깊게 쌓아 파라미터를 늘린 모델 구조를 통해 특성 추출 과정을 알고리즘 내부로 통합(자동화)
 - 사람이 직접 특성을 추출하지 않아도 모델 스스로 데이터의 유용한 패턴을 찾아냄

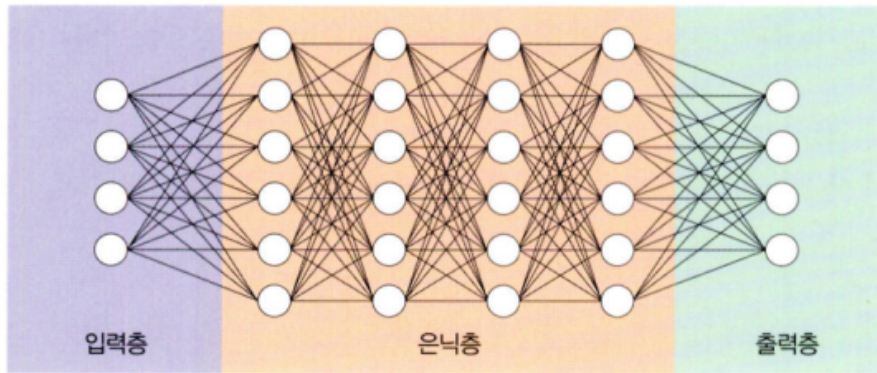
2) 빅데이터의 효율적 활용

- 딥러닝의 자동 특성 추출 능력: **학습 데이터의 양↑ → 성능이 급격히 ↑**
- 적용 기준:
 - 데이터가 풍부한 경우: 딥러닝을 적용할 때 성능 극대화를 기대 가능
 - 데이터가 적은 경우: 딥러닝 성능 향상이 어려움 → 기존 머신러닝 알고리즘을 고려하는 것이 유리

4.3 딥러닝 알고리즘

4.3.1 심층 신경망

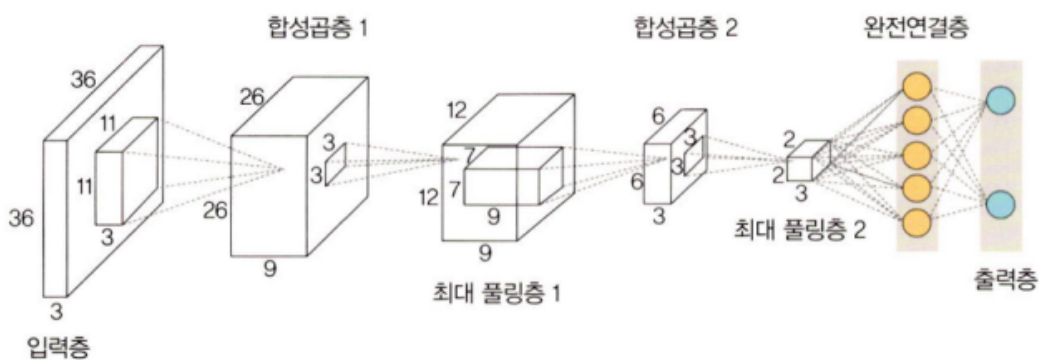
: 입력층과 출력층 사이에 다수의 은닉층을 포함하는 인공 신경망



장점	머신러닝과 다르게 별도의 트릭 없이 비선형 분류 가능
단점	- 학습을 위한 연산량 ↑ - 기울기 소멸 문제 발생 가능
해결 방안	- 드롭아웃 - 렐루 함수 - 배치 정규화

4.3.2 합성곱 신경망(CNN: Convolutional Neural Network)

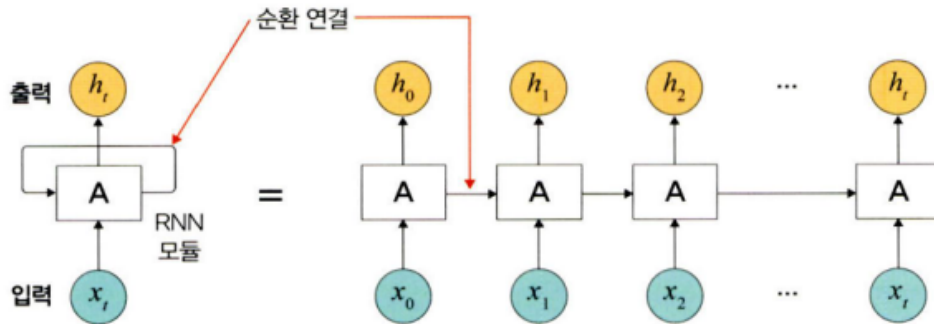
: 합성곱층(Convolutional Layer)과 풀링층(Pooling Layer)을 포함하며, **이미지** 처리 및 **영상** 내 객체 인식/위치 탐색에 뛰어난 신경망



특징	공간 정보 유지: 입출력 데이터의 형상(3D 구조 등)과 이미지 내 위치적/공간적 특징을 유지 특징 추출 & 강화: 복수의 필터 → 이미지의 특징을 추출, 풀링층(Pooling Layer) → 중요한 특징을 집계/강화 파라미터 절감: 필터의 가중치를 공유(Shared Weight) → 일반 인공신경망(ANN) 대비 학습할 파라미터 수(가중치 개수) 훨씬 적음
대표적 모델	- 초기/기본: LeNet-5, AlexNet - 심층 모델: VGG, GoogLeNet, ResNet

4.3.3 순환 신경망(RNN: Recurrent Neural Network)

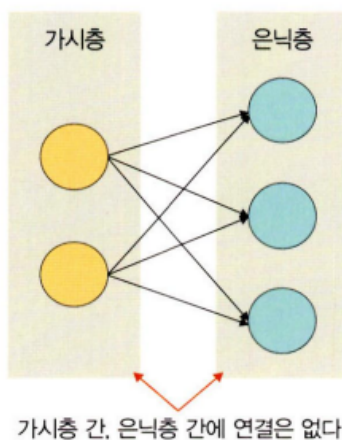
: **시간의 흐름**에 따라 변화하는 시계열 데이터(Temporal Data)나 순차 데이터(Sequential Data)를 학습하기 위해 자기 자신을 **순환(Recurrent)** 참조하는 구조의 신경망



특징	• 시간성(Temporal Property): 이전 시점의 출력 결과가 다음 시점의 입력으로 연결되어 과거 정보(시간성)를 함께 반영 • 동적/가변적 데이터: 시간에 따라 내용이 변하므로 데이터 길이가 가변적인 특성을 가짐
한계	텍스트나 입력이 길어지면 기울기 소멸 문제(Vanishing Gradient)가 발생
대안	→ LSTM(Long Short-Term Memory) 구조가 주로 사용됨
주요 활용 분야	• 자연어 처리(NLP) • 기계 번역 • 텍스트 생성 • 음성 인식 • 이미지 캡셔닝

4.3.4 제한된 볼츠만 머신(RBM: Restricted Boltzmann Machine)

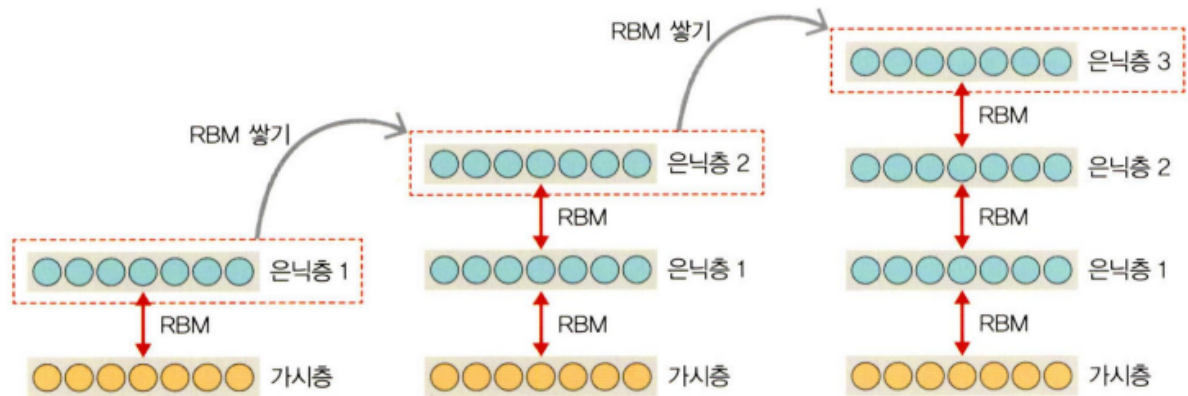
: 가시층(Visible Layer)과 은닉층(Hidden Layer)의 2개 층으로만 구성된 무방향 이분 그래프 형태의 확률 모델



특징	• 제한의 의미: 동일 층 내부(가시층-가시층, 은닉층-은닉층) 간에는 상호 연결이 존재X • 실무에서는 CNN, RNN에 비해 활용 빈도↓
용도	차원 축소, 비지도 학습 기반 특성 추출, 협업 필터링, 심층 신뢰 신경망(DBN)의 기본 블록으로 활용

4.3.5 심층 신뢰 신경망(DBN: Deep Belief Network)

: 사전 훈련된 **제한된 볼츠만 머신(RBM)**을 블록처럼 **수직으로 여러 층** 쌓아 올린 형태의 신경망
 → 비지도 학습을 통해 고차원 추상 특성을 추출하고 다층 퍼셉트론(MLP)의 최적 가중치 초기값을 제공



특징	• RBM 블록 기반의 순차적 학습 ◦ 사전 훈련된 RBM을 아래에서부터 위로 한 층씩 순차적으로 쌓아 올림 → 계층적 구조 생성 • 비지도 학습 가능 ◦ 정답 레이블 X 데이터 → 데이터 자체의 확률 분포와 패턴을 스스로 학습 • 단계별 계층적 추상화 (Feature Abstraction) ◦ 위쪽 층으로 올라갈수록 단순한 요소에서 점차 고차원의 추상적인 특성을 추출 ◦ 부분적인 이미지/정보만 보고 전체를 연상하고 복원하는 일반화 및 추상화 작업에 유용 • MLP 가중치 초기값 제공 (기울기 소멸 완화) ◦ DBN을 통해 사전 학습된 가중치를 다층 퍼셉트론(MLP)의 가중치 초기값으로 설정 ◦ 무작위 초기값을 사용으로 인한 기울기 소멸 문제를 완화 + 이후 진행할 지도 학습의 성능과 수렴 속도를 크게 ↑
학습 절차	1. 가시층과 은닉층 1에 RBM을 사전 훈련 2. 첫 번째 층 입력 데이터와 파라미터를 고정 → 두 번째 층 RBM을 사전 훈련 3. 원하는 층 개수만큼 RBM을 쌓아 올려 DBN을 완성